

Foveated VR Rendering System for Large 3D Meshes

Huadong Zhang*

School of Interactive Games and Media
Rochester Institute of Technology, USA

Chao Peng†

School of Interactive Games and Media
Rochester Institute of Technology, USA

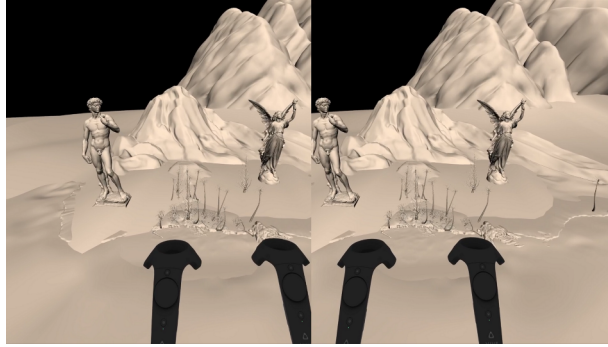


Figure 1: Rendering of a complex scene on a Quest Pro, showing left and right eye views. The scene includes partial Moana Island, and the David and Lucy models, composed of 95,967,582 triangles and 48,487,571 vertices, and rendered at an average of 113 FPS.

Index Terms: Foveated LOD selection, massive model rendering.

1 INTRODUCTION

Real-time rendering technology is crucial for exploring complex 3D meshes. With increasing demand for realistic virtual experiences in fields such as digital heritage, architecture, and scientific visualization, VR applications now require high-fidelity models and efficient rendering of scenes with millions of vertices to deliver convincing immersion and self-presence.

Rendering large meshes in VR headsets intensifies performance requirements due to the need for high frame rates (90 fps) to ensure comfort and avoid nausea, stereo rendering that doubles the computational workload, and a wider field of view (over 100 degrees) that demands rendering more of the scene at high detail.

To speed up VR rendering performance, this work integrates a foveated approach into LOD selection, preserving high detail in the foveal region while reducing detail in peripheral areas to lower rendering load. A hierarchical patch-based mesh structure enables accurate load estimation and continuous LOD adjustments, with coarser details at higher hierarchy levels. Figure 1 shows a VR rendering example with our approach for a complex 3D scene.

2 HIERARCHICAL MESH STRUCTURE

Our method employs a hierarchical structure to optimize VR rendering by directing computational resources to the most visually important areas. We use graph partitioning [4] and spatial partitioning [3] techniques to construct the mesh hierarchy, where each node represents a distinct part of the original mesh at varying levels of detail. The nodes are uniformly sized, simplifying the estimation of rendered triangles and allowing dynamic adjustments based on their visual significance.

*e-mail: hz2208@rit.edu

†e-mail: cxpigm@rit.edu

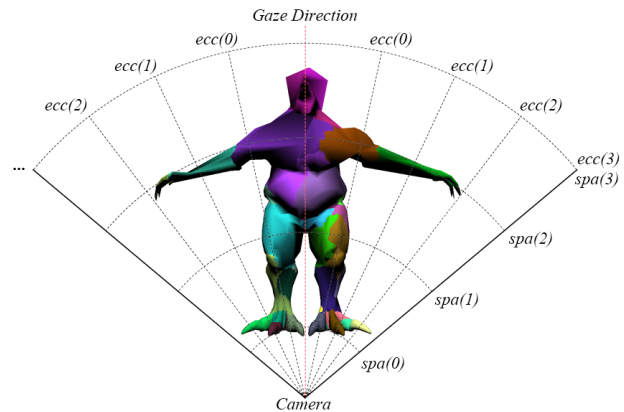


Figure 2: An example of parametric space for LOD selection with a mesh hierarchy consisting of four levels. $spa()$ and $ecc()$ represent ranges in distance and angle, respectively, for the level heights in the mesh hierarchy at 0, 1, 2, and 3.

Using the Progressive Meshes technique [2], edges within each node are continuously collapsed, reducing the number of triangles and simplifying the topology. This process progressively decreases the geometric detail of each node. Parent nodes represent simplified versions of their child nodes, reducing complexity by approximately 50%. Consequently, Higher hierarchy levels correspond to simpler mesh representations, enabling both progressive simplification across levels and within individual nodes.

3 FOVEATED LOD SELECTION

Our LOD selection criteria include both spatial factors and foveal focus, aiming to select a subset of nodes that maintain high geometric detail in visually important areas.

Bounding spheres are used to estimate the spatial size of nodes on the screen. Nodes with larger bounding spheres, which occupy a larger screen area, are considered more visually important. As the camera moves away, these larger bounding spheres are less prone to being replaced by higher-level nodes. Conversely, as the camera moves closer, nodes with larger bounding spheres are more inclined to be refined to lower-level nodes compared to smaller ones.

The foveal region is a small, central portion of the retina. It is represented by small values of eccentricity, which is defined as the angular distance away from the ventral gaze direction. When measuring the eccentricity, we look for equalizing the effect of changes across different bounding volume sizes, so that they can be evaluated under a common threshold. To do this, for each node, we identify the point on its bounding sphere that forms the smallest angle with the gaze direction. If two nodes have the same geometric complexity but differ in size, their values of eccentricity will be identical if the angles formed by their closest points to the gaze direction are the same.

We define a parametric space using spatial distance and angular eccentricity, divided into selection zones based on distance and angle dimensions. Boundary values correspond to the leaf (finest detail) and root (coarsest detail) node levels. Each node is eval-



Figure 3: An example of a foveated LOD selection result. The white dots represent the gaze direction. Rendering quality decreases gradually as it moves away from the gaze point.

uated to determine whether it should be selected or adjusted to a coarser or finer detail level. Figure 2 illustrates this selection process. This parametric space is view-dependent, aligning with the camera's depth and field of view.

The node selection process starts at the leaf nodes and traverses the hierarchy bottom-up. All leaf nodes are initially considered for selection. Nodes are then tested in parallel against the parametric space criteria, and those failing to meet the criteria are deselected. Sibling nodes are evaluated next; if both are deselected, their parent node is selected, otherwise, both siblings remain selected. This process repeats at each level until reaching the root node. Figure 3 shows an example of a foveated LOD selection result.

The node selection process generates a set of nodes for adaptive meshes based on spatial factors and foveal focus. In performance-constrained scenarios, where the number of selectable nodes is fixed, exceeding the load budget can lead to memory overflow. To prevent this, a pruning strategy replaces sibling nodes with higher eccentricity and lower spatial importance with their parent nodes, ensuring the node count stays within budget while minimizing visual quality loss.

4 MESH REFORMATION AND RENDERING

For each selected node, we want to represent it with a desired level of primitive details determined by a specific number of vertices and triangles. As discussed in Section 2, the nodes represent regions on the mesh surface, each populated with a varying density of vertices and triangles. This granularity has been established during pre-processing, ranging from its most detailed representation—featuring the maximum vertices and triangles—to a coarser form, derived through progressive edge-collapsing techniques.

We determine the geometric details for each selected node based on eccentricity and spatial values within the LOD selection's parametric space, mapped to a normalized scale. A value of 0 corresponds to the coarsest level, 1 represents the finest level, and intermediate values linearly correspond to the desired number of vertices and triangles. These values are multiplied to compute a ratio factor that determines the node's geometric details. Simplified nodes are constructed at run-time using a GPU-based mesh reformation method [1, 5] with triangle-level parallelization.

To further optimize performance, we implemented a parallelized culling process at the node level. For rasterization, we adopted the strategy that uses the single buffer object with multiple draw calls, rather than issuing separate buffer objects for each node. This approach significantly reduces CPU overhead associated with buffer switching, thereby enhancing overall rendering efficiency.

5 EVALUATION

We implemented our VR rendering system using OpenVR, OpenGL, and CUDA 11.6. Run-time computations were performed on a workstation with an Intel Core i9-10980XE CPU (3.00 GHz), 128 GB RAM, and an NVIDIA GeForce RTX 3090 GPU (24 GB), running 64-bit Windows 10 with NVIDIA driver version 546.01. Real-time rendering was displayed on a Oculus Quest Pro headset.

We conducted a performance breakdown to examine the efficiency of our system's major components: LOD selection, culling, reformation, and rasterization. Rasterization dominates the overall rendering time, taking 63.45% of the frame render time. In comparison, the LOD selection, culling, and reformation stages required only 4.45%, 0.73%, and 31.37% of the render time, respectively. The relatively low processing time for LOD selection, culling, and reformation demonstrates the efficiency of these components in optimizing the rendering pipeline. Despite rasterization being the primary performance bottleneck, the overall performance remains strong due to the efficiency of other components.

We compared the performance of our system with widely-used rendering solutions, including the OpenVR OpenGL framework, Unity URP (Universal Render Pipeline), Unity HDRP (High Definition Render Pipeline), and Unreal Engine's Nanite. The Unity tests were conducted using version 2022.3.7f1, while Unreal Engine tests were performed on version 5.3.2. Unreal Engine's Nanite utilizes a hierarchical level-of-detail system, which dynamically selects and streams the necessary details based on the viewer's perspective. Nanite is highly effective in VR rendering, where it optimizes performance by reducing the load of high-detail geometry while maintaining visual fidelity in areas closest to the viewer. Unity URP is designed for scalability and real-time performance across a wide range of platforms including VR, while Unity HDRP focuses on delivering high-fidelity visuals with slightly higher performance costs. Both URP and HDRP support VR applications very well. Our system achieves an average rendering time of 8.85 ms per frame, significantly outperforming the other solutions. This is 2.63 times faster than Unreal Engine's Nanite, 3.06 times faster than URP, and 3.13 times faster than HDRP. Additionally, our method demonstrates the least fluctuation in performance. The standard OpenVR framework exhibits the slowest performance due to its lack of advanced mesh structures for data representation and no optimization in the geometric processing stage.

6 CONCLUSION

This paper presents a method for foveated VR rendering of polygonal meshes, designed to efficiently handle large-scale, high-detail scenes on the GPU. By focusing on the foveal region, where visual acuity is highest, the approach ensures computational resources are directed to the most perceptually significant areas. A uniform and balanced hierarchical mesh structure enables load estimation and efficient resource allocation. The method employs a parametric space that integrates spatial distance and angular eccentricity, prioritizing key regions while reducing detail in peripheral areas.

REFERENCES

- [1] Y. Dong and C. Peng. Multi-gpu multi-display rendering of extremely large 3d environments. *The Visual Computer*, 39(12):6473–6489, 2023.
- [2] H. Hoppe. Progressive meshes. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, pp. 111–120. 2023.
- [3] T. Karras and T. Aila. Fast parallel construction of high-quality bounding volume hierarchies. In *Proceedings of the 5th High-Performance Graphics Conference*, pp. 89–99, 2013.
- [4] G. Karypis and V. Kumar. Metis: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices. 1997.
- [5] C. Peng and Y. Cao. A gpu-based approach for massive model rendering with frame-to-frame coherence. In *Computer Graphics Forum*, vol. 31, pp. 393–402. Wiley Online Library, 2012.